Task 12

Step 1: Create EC2 with Auto Scaling Group

1. Create a **Launch Template**:
    a. AMI: Amazon Linux 2
    b. Instance type: t2.micro (or equivalent)
    c. Attach an **IAM Role** with:
        i. CloudWatch access
        ii. EC2 access (read/write)
        iii. SNS publish permission
    d. Install a simple web server (Apache/Nginx)
2. Create an **Auto Scaling Group (ASG)**:

This ensures EC2 can be replaced automatically.

Step 2: Configure CloudWatch Monitoring

1. Go to **CloudWatch → Alarms**
2. Create an alarm based on **EC2 metric**:

This alarm represents "failure detection".

Step 3: Create SNS Topic for Notifications

1. Go to **SNS → Topics**
2. Create a standard topic (example: `ec2-self-healing-alerts`)
3. Add email subscription(s)
4. Confirm the subscription from your email inbox

SNS will notify when failure and recovery occur.

Step 4: Create Lambda Function (Self-Healing Logic)

Lambda is the **automated fixer**.

Step 5: Connect CloudWatch Alarm to Lambda

1. Edit the **CloudWatch Alarm**
2. Set alarm action:

    a. Trigger the **Lambda function**
3. Save the alarm

This connects detection → action.

Step 6: Intentionally Fail the EC2 Instance

Use **any ONE** of the following:

Option 1 (Recommended):

- Stop the EC2 instance from AWS Console
  Do NOT manually recover the instance.

Step 7: Observe Self-Healing Behavior

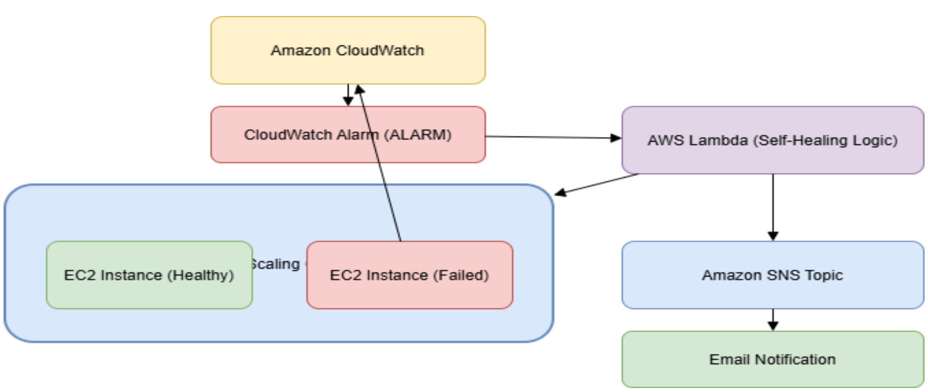
Verify the following:

- CloudWatch alarm changes to **ALARM**
- Lambda function is invoked
- Auto Scaling replaces or recovers EC2
- SNS notification email is received
- New EC2 instance becomes healthy

Output:

EC2 failure must be detected, fixed, and notified **automatically**, without any manual intervention.

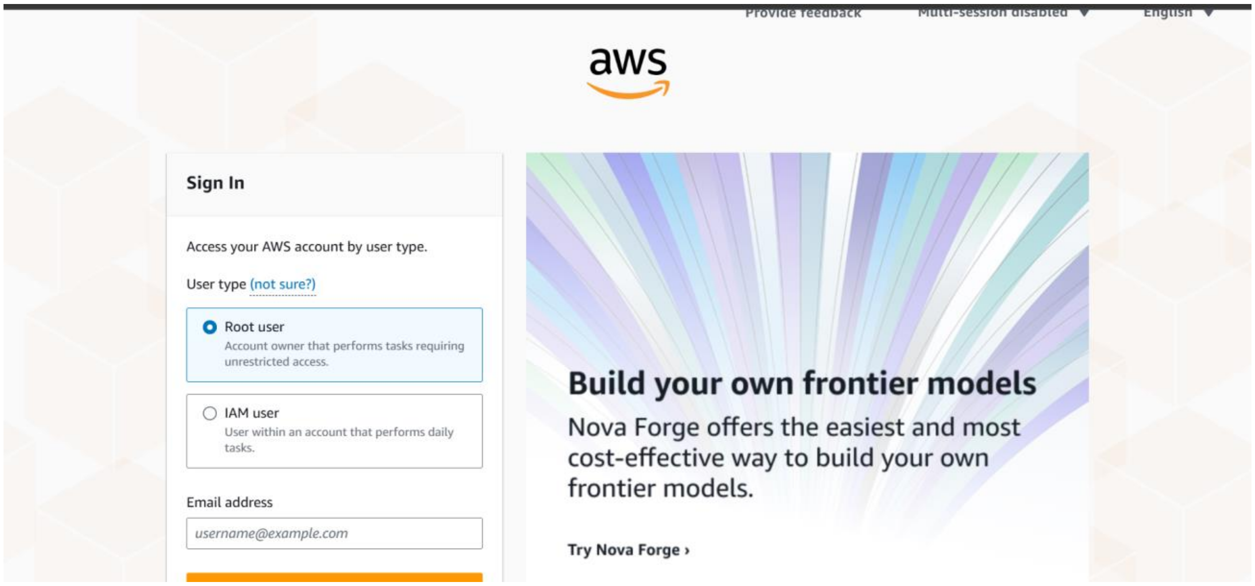
Workflow:



Solution:

1. Login to your AWS account with credentials and required verification.

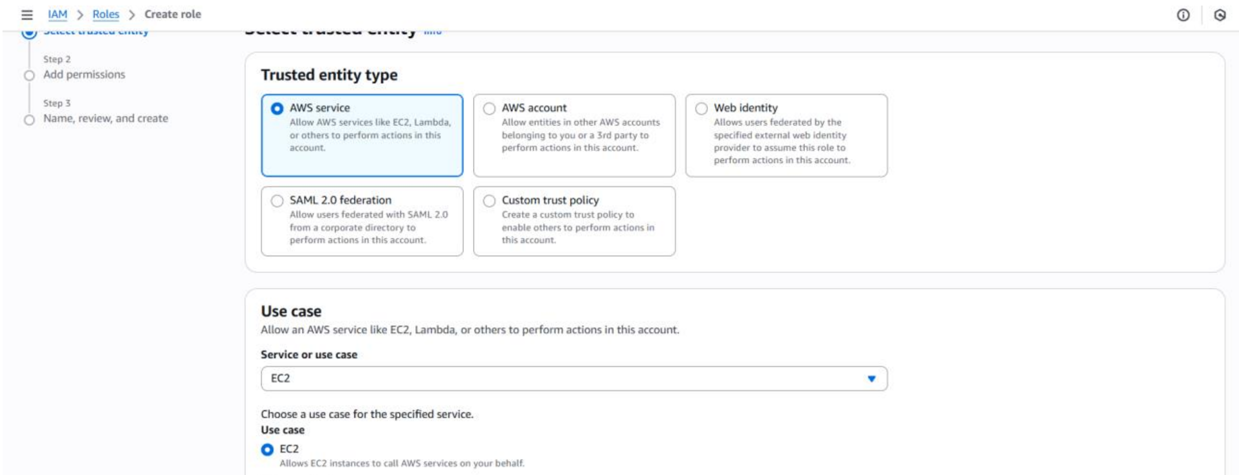
   (https://signin.aws.amazon.com/)
   My account – AWS Management Console- Sign in

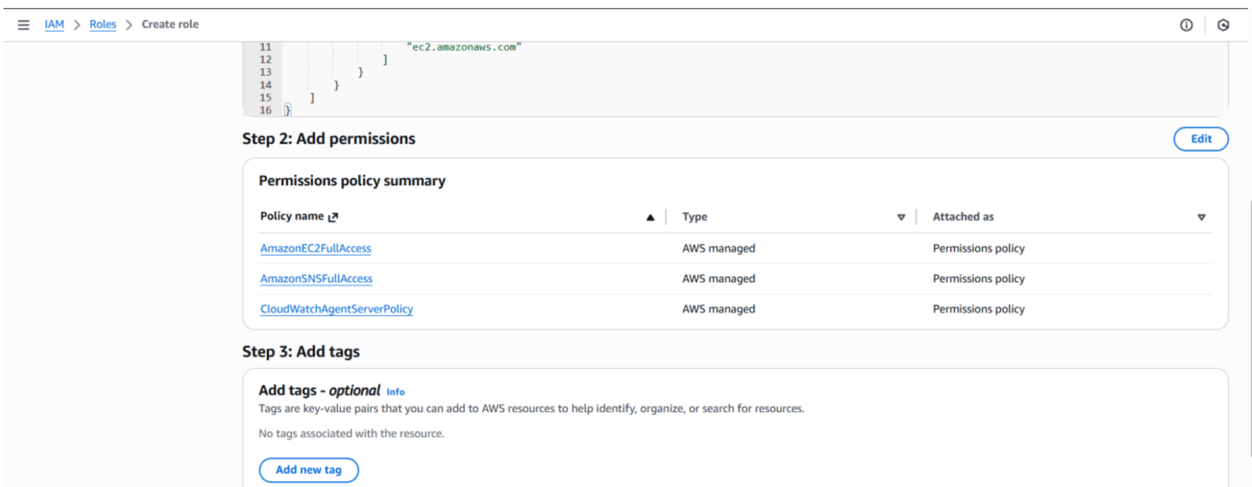


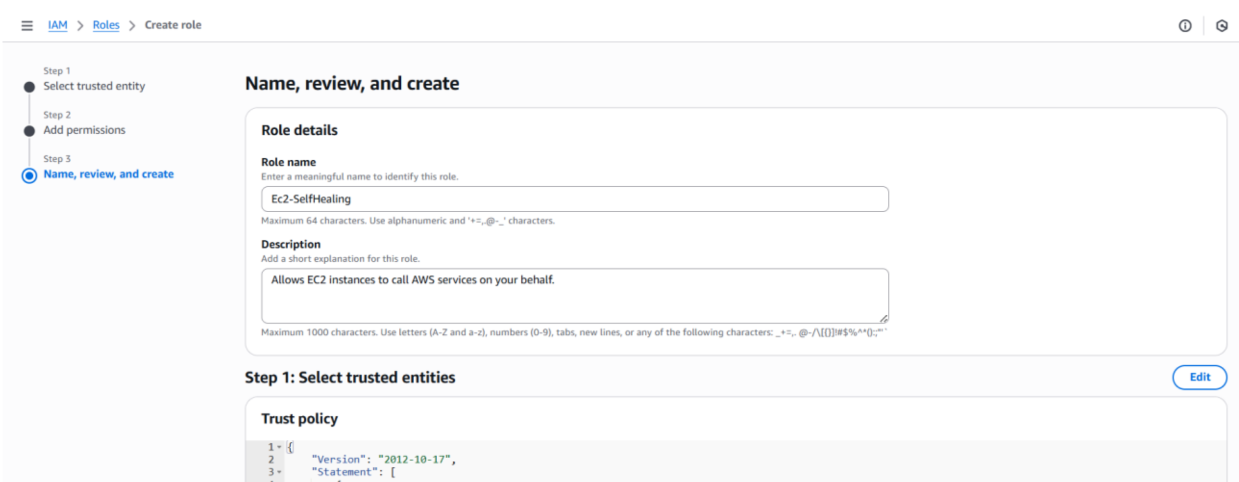
   Create Auto Scaling Group

2. For Auto Scaling we need to create an IAM Role

- Go to IAM - Roles - Create role
- Trusted entity:
    - Select AWS service
    - Choose EC2
- Attach policies:
    - *CloudWatchAgentServerPolicy*
    - *AmazonEC2FullAccess*
    - *AmazonSNSFullAccess*
- name: *Ec2-SelfHealing*
- Create role.







3. Create Launch Template
   - Go to EC2 - Launch Templates - Create launch template
   - name: *Self-Healing-Template*

- AMI:
    - Amazon Linux 2
- Instance type:
    - t2.micro
- Key pair:
    - Select or create one
- Network settings:
    - Allow HTTP (port 80) and SSH (port 22)
- IAM instance profile:
    - Select *EC2-SelfHealing*
- User Data (Install Apache)

    #!/bin/bash

    yum update -y

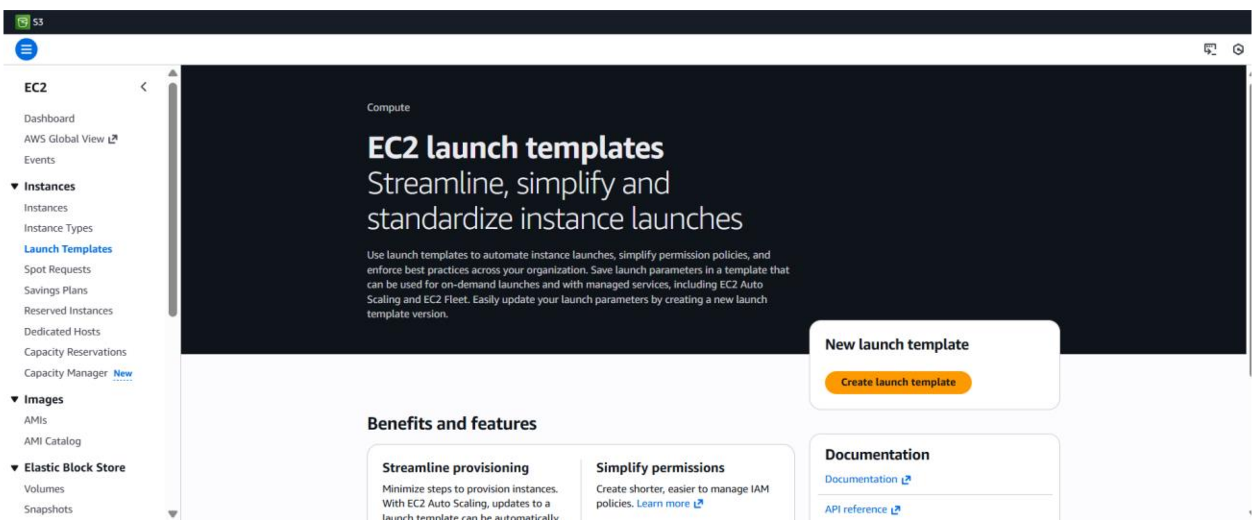
    yum install httpd -y

    systemctl start httpd

    systemctl enable httpd

    echo "Self Healing EC2 is running" > /var/www/html/index.html

- Click Create launch template



Recents | **Quick Start**

Don't include in launch template	Amazon Linux	macOS	Ubuntu	Windows	Red Hat	SUSE Linux	Debian
	aws	Mac	ubuntu	Microsoft	Red Hat	SUSE	debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI — Free tier eligible
ami-0ced6a024bb18ff2e (64-bit (x86), uefi-preferred) / ami-01605b62a13170a53 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.10.20260105.0 x86_64 HVM kernel-6.1

Architecture	Boot mode	AMI ID	Publish Date	Username	
64-bit (x86)	uefi-preferred	ami-0ced6a024bb18ff2e	2026-01-02	ec2-user	Verified provider

EC2 > Launch templates > Create launch template

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

self-healing-template

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

A prod webserver for MyApp

Max 255 chars

Auto Scaling guidance | Info

Select this if you intend to use this template with EC2 Auto Scaling

☐ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

▶ Template tags

▶ Source template

Launch template contents

Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

▼ **Summary**

Software Image (AMI)
-

Virtual server type (instance type)
-

Firewall (security group)
-

Storage (volumes)
-

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier allowance.

Cancel | Create launch template

▼ Instance type Info | Get advice

Advanced

Instance type

t2.micro
Family: t2 1 vCPU 1 GiB Memory Current generation: true On-Demand Windows base pricing: 0.017 USD per Hour
On-Demand RHEL base pricing: 0.0268 USD per Hour On-Demand Linux base pricing: 0.0124 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.0142 USD per Hour On-Demand SUSE base pricing: 0.0124 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

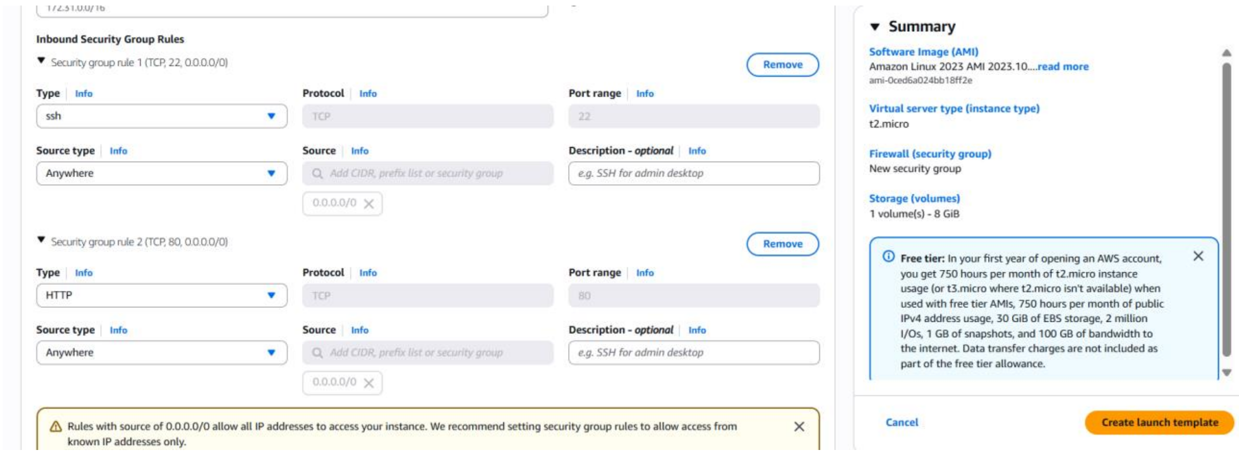
▼ Key pair (login) Info

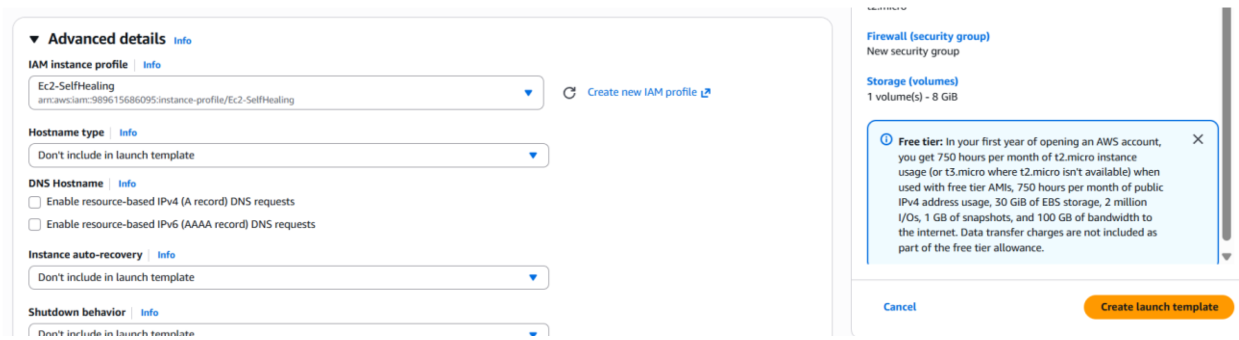
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

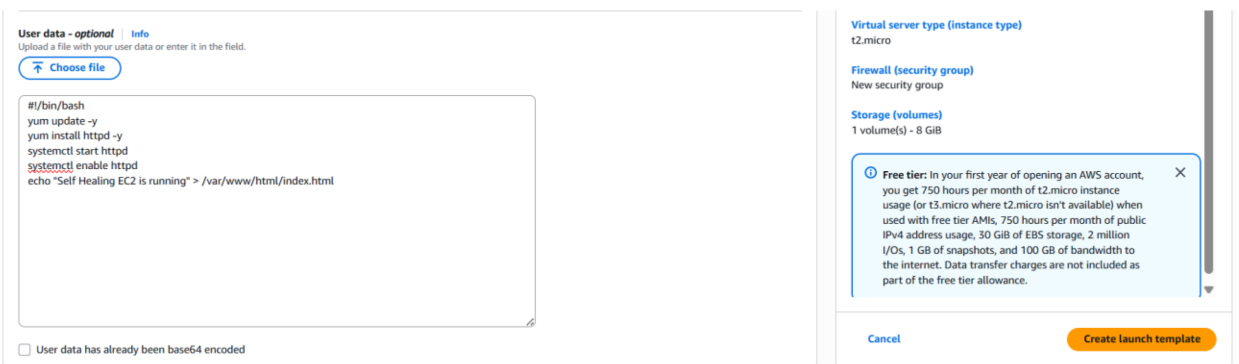
Key pair name

mykey

Create new key pair



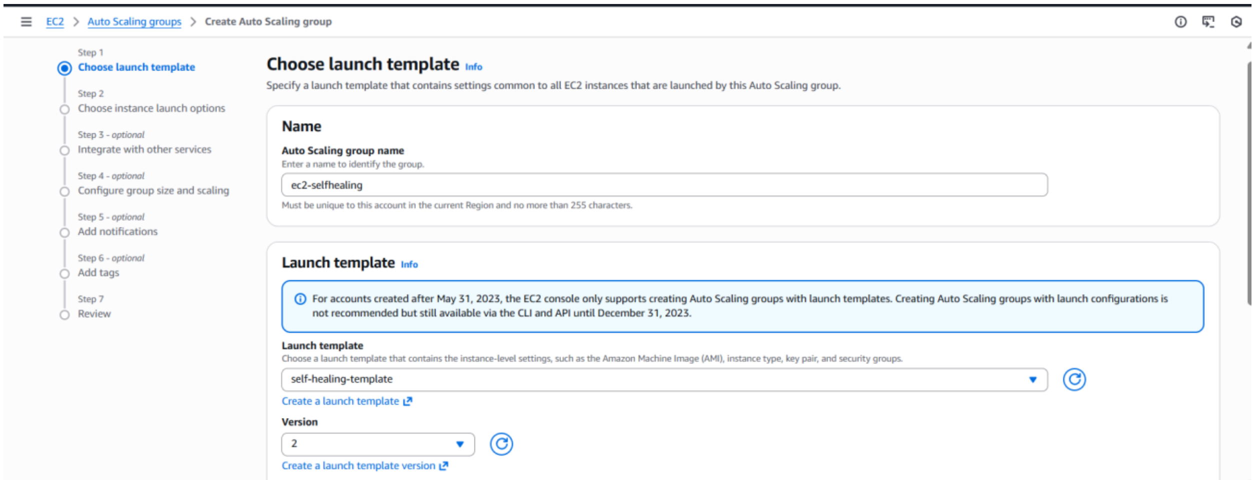


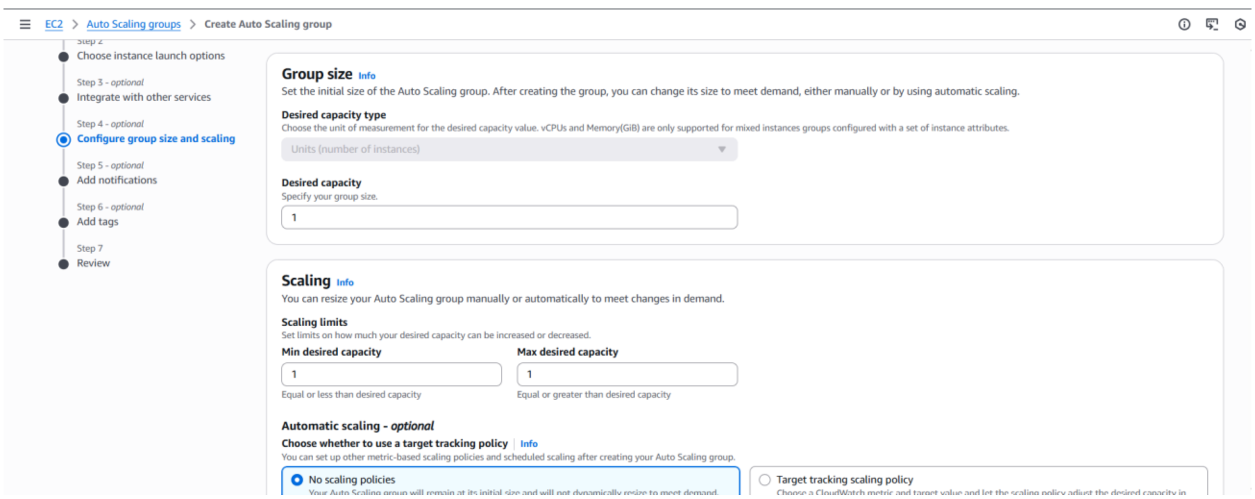


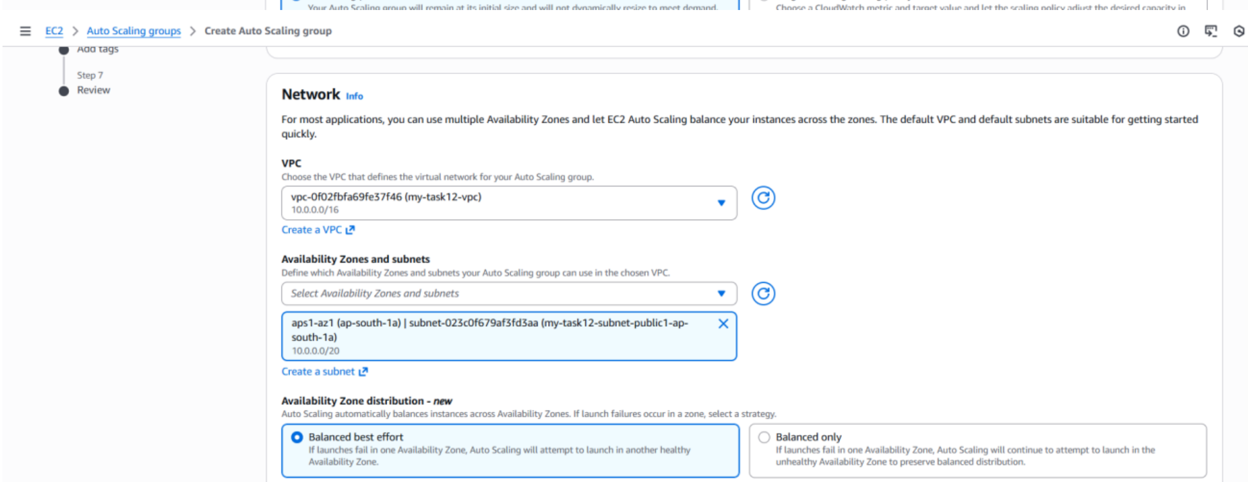
4. Create Auto Scaling Group (ASG)

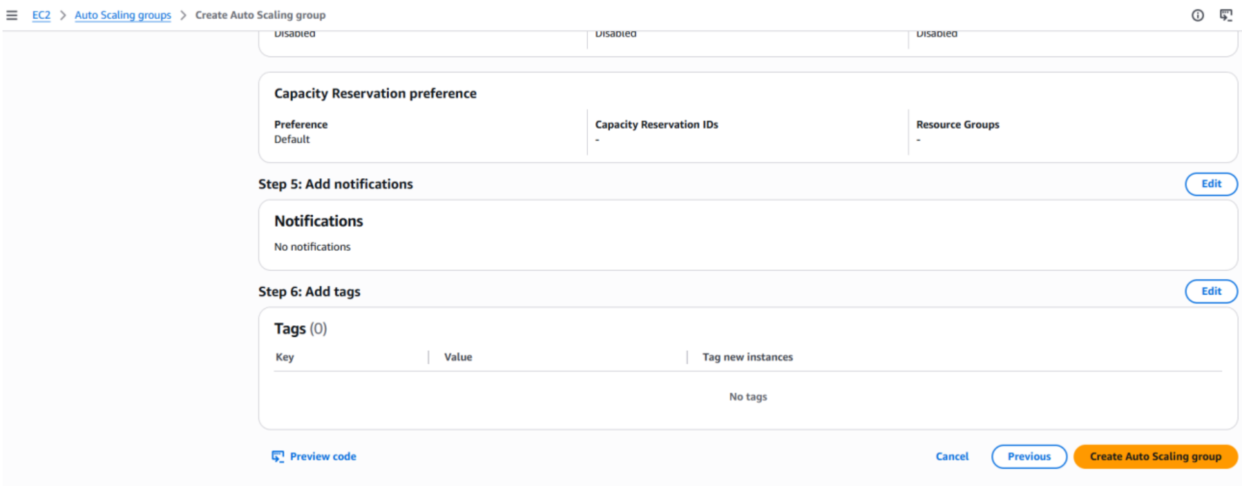
- Go to EC2 - Auto Scaling Groups - Create
- Select launch template:
  - self-healing-template
- Choose VPC and subnet
- Group size:
  - Desired: 1
  - Minimum: 1
  - Maximum: 1
- Health checks:

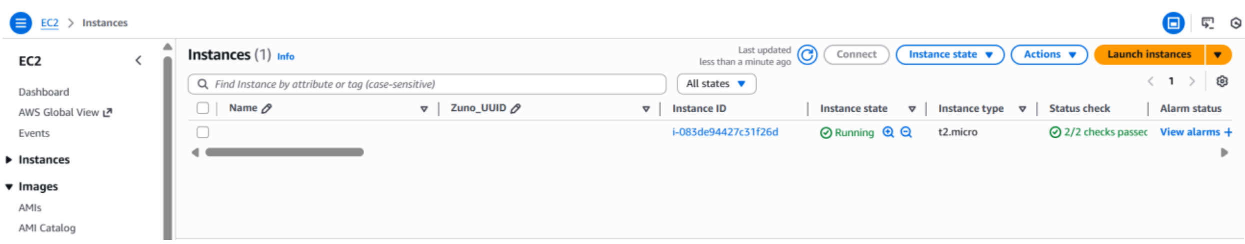
    - Type: EC2
- Keep policies empty
- Create ASG









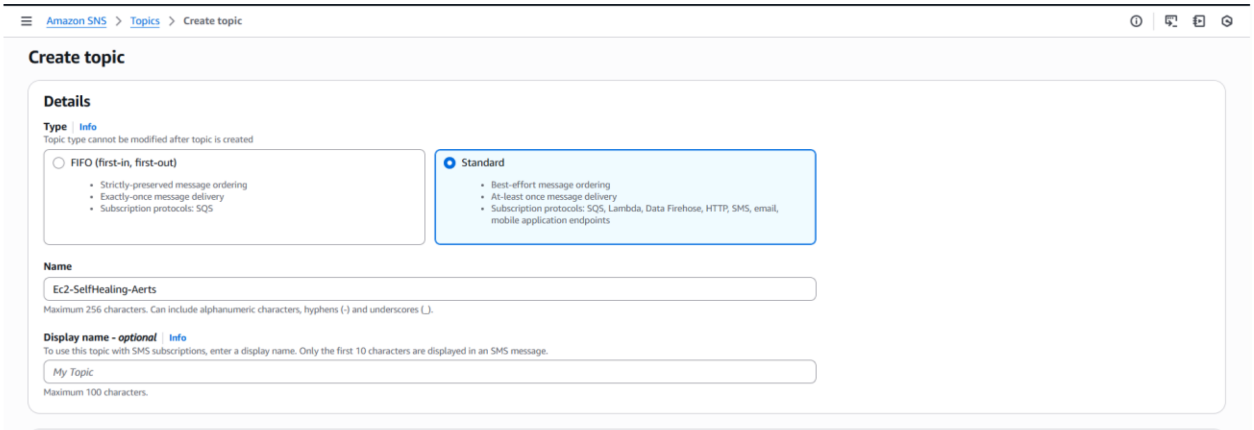


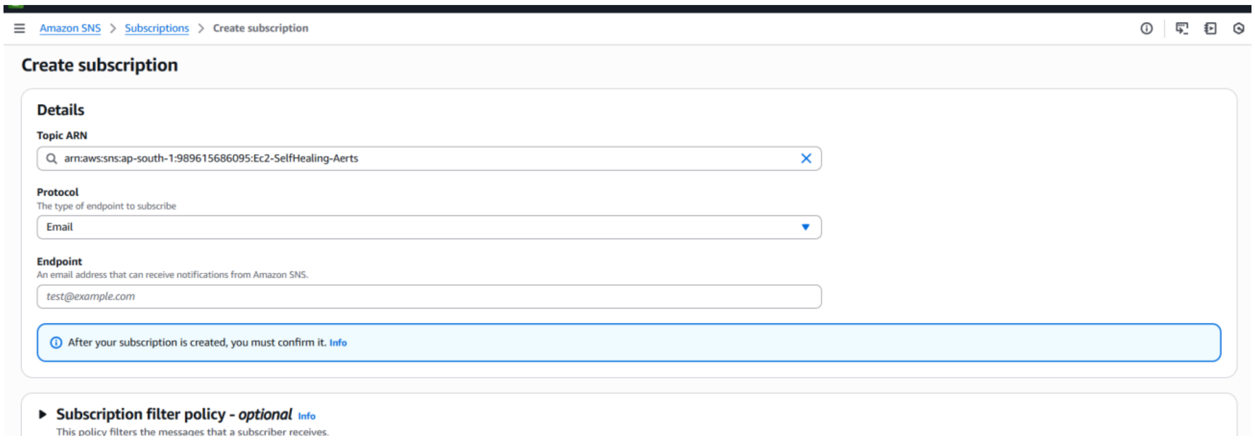
Configure Cloudwatch

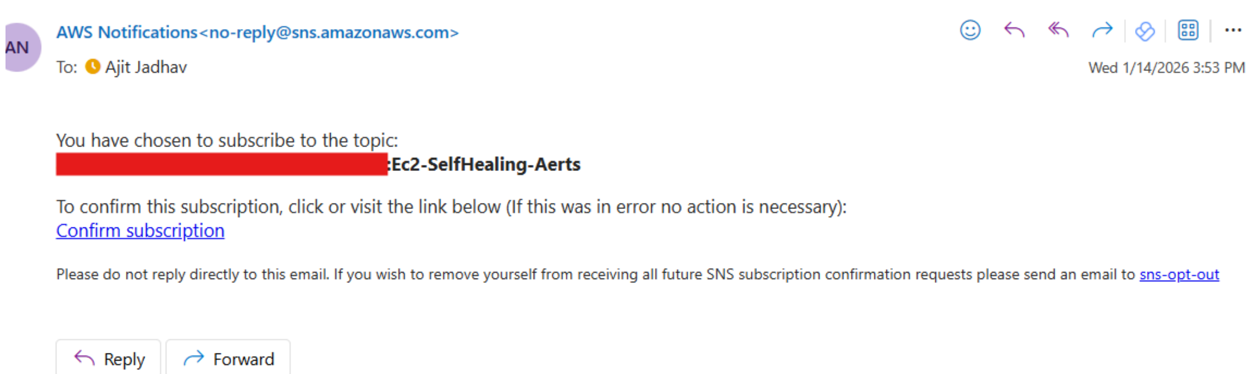
5. Go to SNS - Topics - Create topic

- Type:
    - Standard
- Name:
    - ec2-self-healing-alerts
- Create topic

  Add Email Subscription
- Click Create subscription
- Protocol:
    - Email
- Endpoint:
    - email address
- Create subscription
- Confirm from your email inbox







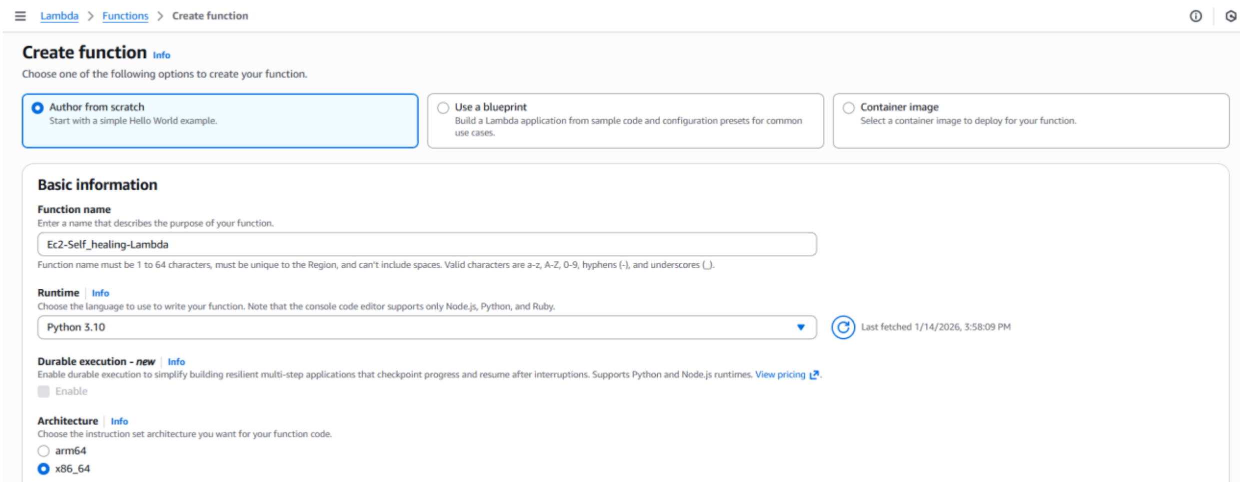
6. Create Lambda Function

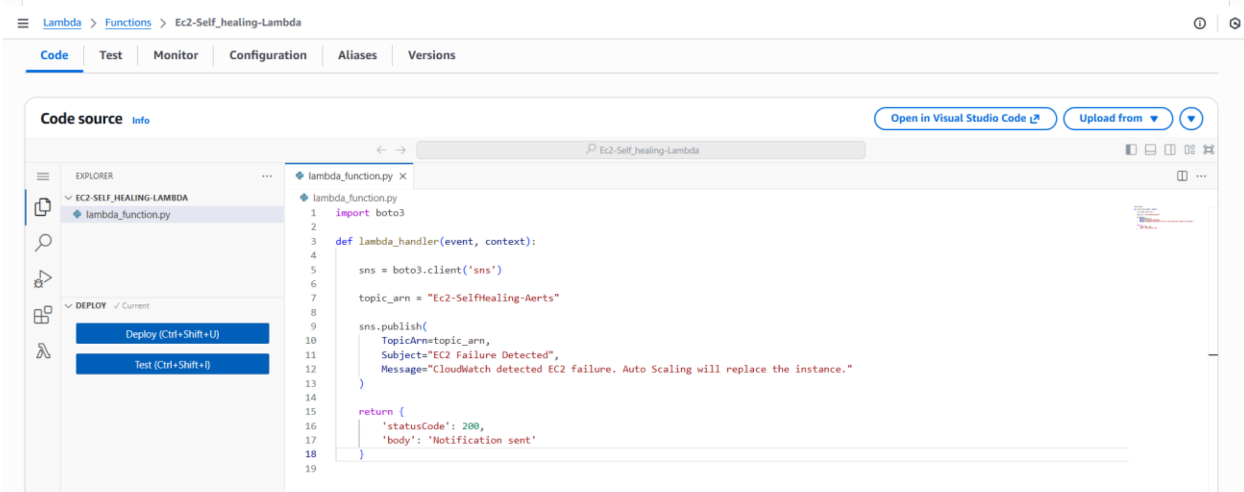
- Go to Lambda - Create function
- Choose:
    - Author from scratch
- Function name:
    - EC2-SelfHealing-Lambda
- Runtime:
    - Python 3.9
- Execution role:

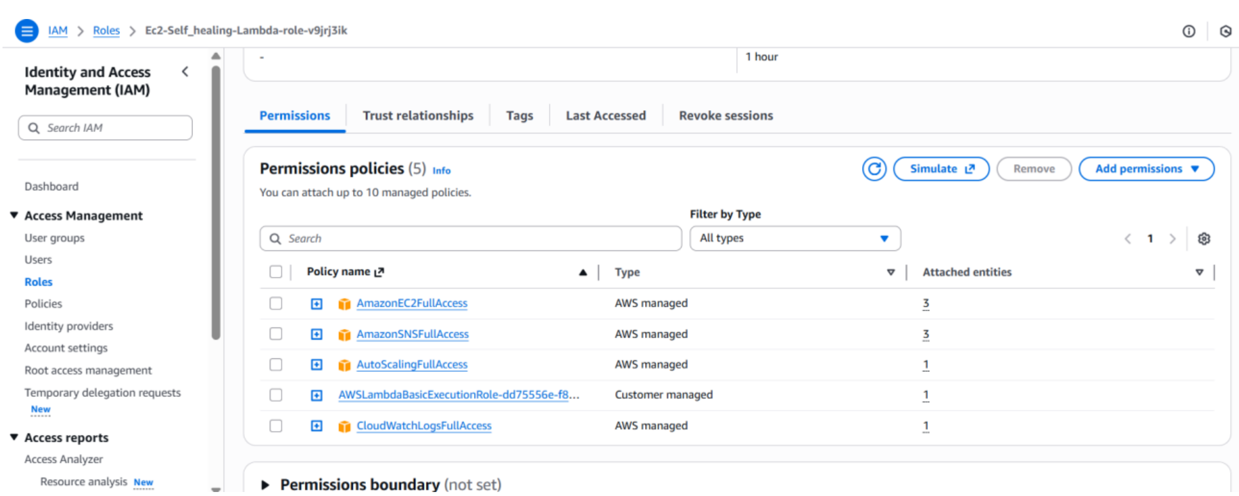
    - Create new role with basic permissions
- Go to IAM - Roles -Lambda role
- Attach:
    - *AmazonEC2FullAccess*
    - *AmazonAutoScalingFullAccess*
    - *AmazonSNSFullAccess*
    - *CloudWatchLogsFullAccess*

Logic Code(Get from Chatgpt)

```
import boto3

def lambda_handler(event, context):

    sns = boto3.client('sns')

    topic_arn = "Ec2-SelfHealing-Aerts"

    sns.publish(
        TopicArn=topic_arn,
        Subject="EC2 Failure Detected",
        Message="CloudWatch detected EC2 failure. Auto Scaling will
replace the instance."
    )

    return {
        'statusCode': 200,
        'body': 'Notification sent'
    }
```



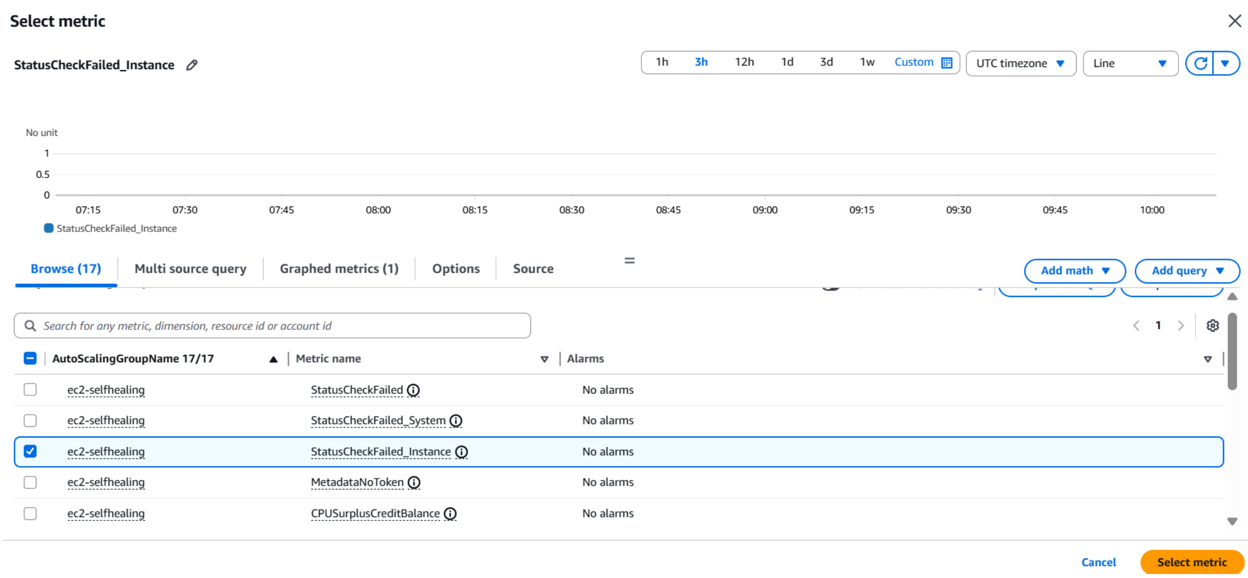


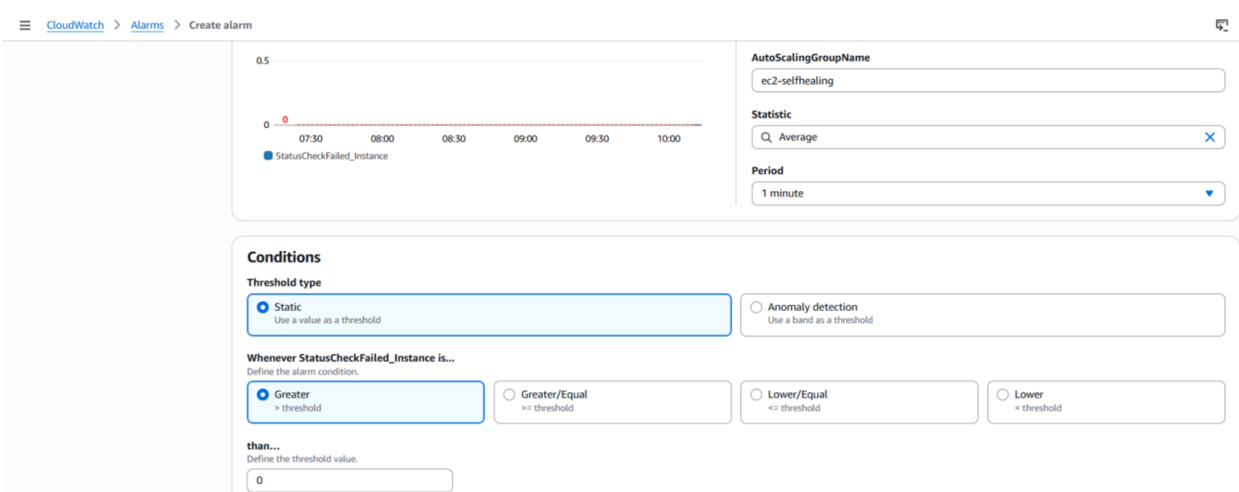


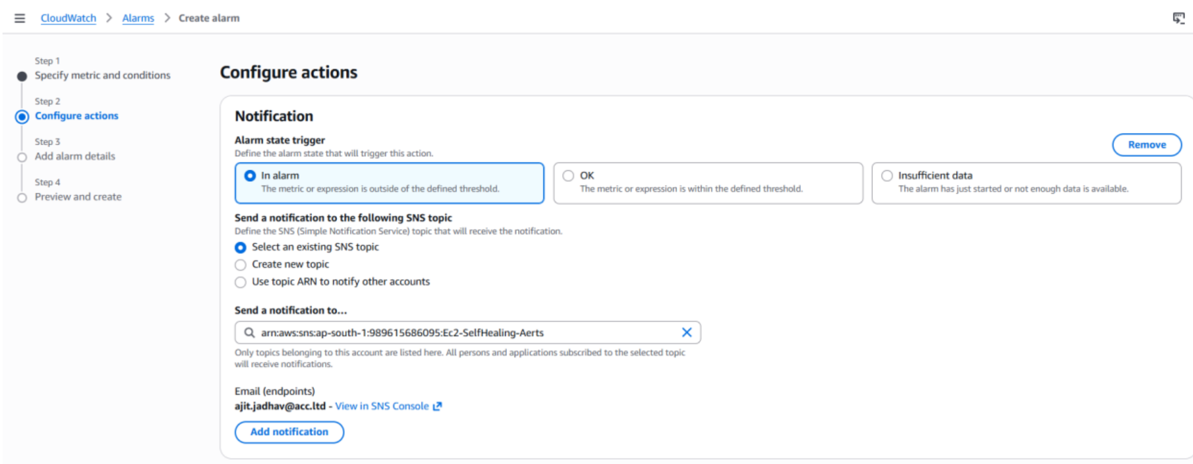
7. Create Alaram

- Go to CloudWatch - Alarms - Create alarm
- Select metric:

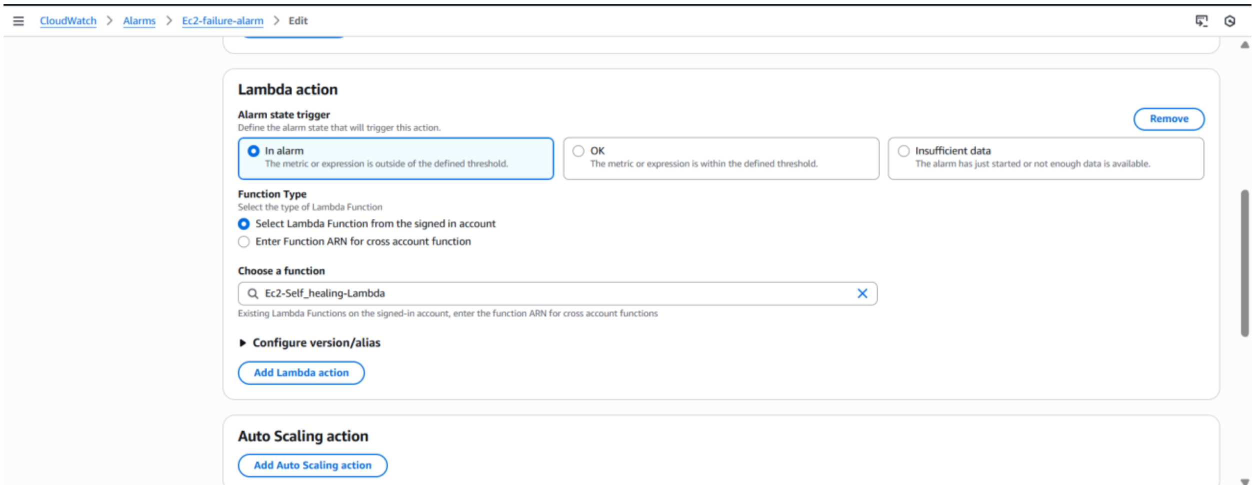
   - EC2 – By Auto Scaling Group
- Choose : *StatusCheckFailed_Instance*
- Conditions:
   - Threshold: Greater than 0
   - Period: 1 minute
   - Evaluation periods: 1
- Alarm name: EC2-Failure-Alarm







Add lambda function



Test :

- Go to EC2 - Instances
- Select the EC2 instance created by ASG
- Click Instance state - Stop
- Do not restart it

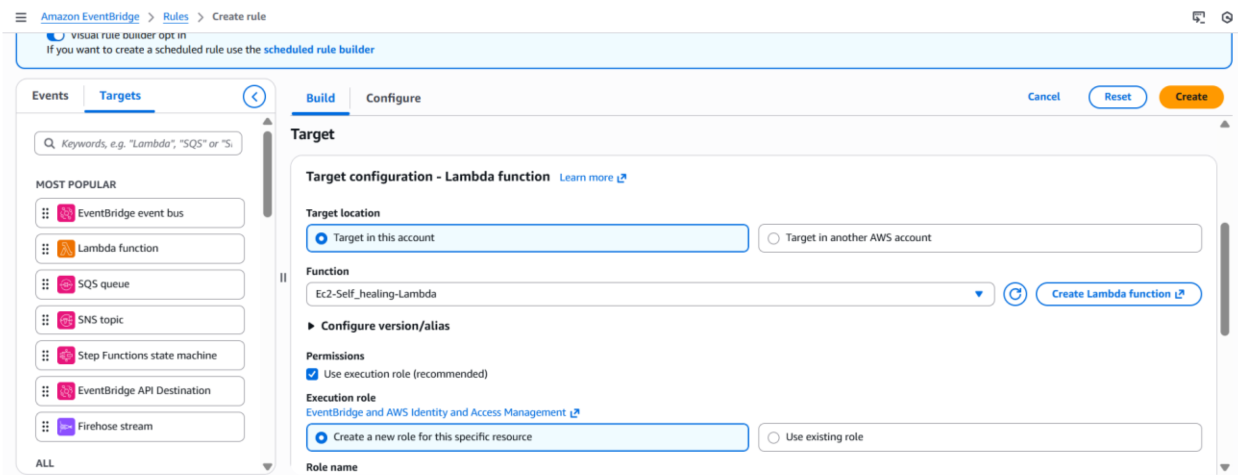
Observe

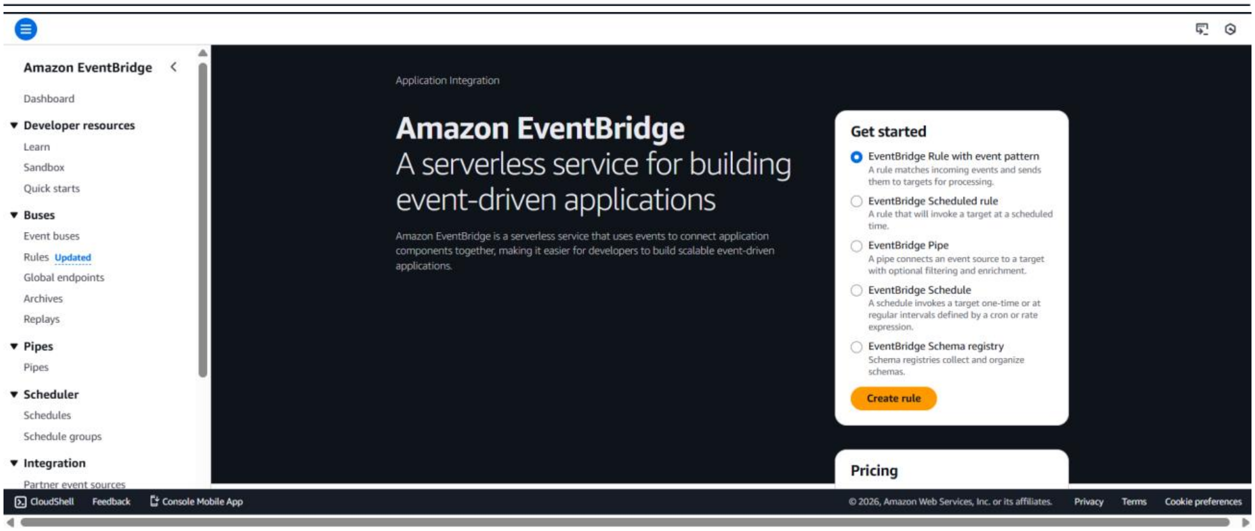
While setting up Realize that the process o manually doing to instances like stop/terminate Is not detected by Alarm,
Because Alram only detect internal process like OS crash, Status Unhealthy etc

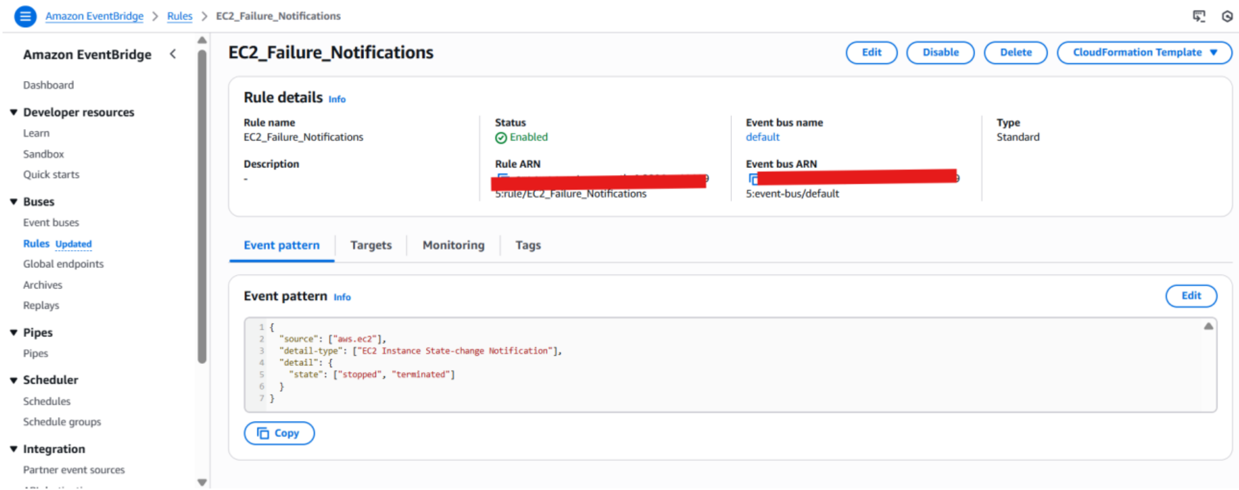
So now we will set a Eventbridge rule to check changing of states of instances

So, thus will detect our instance state and trigger our lambda function.

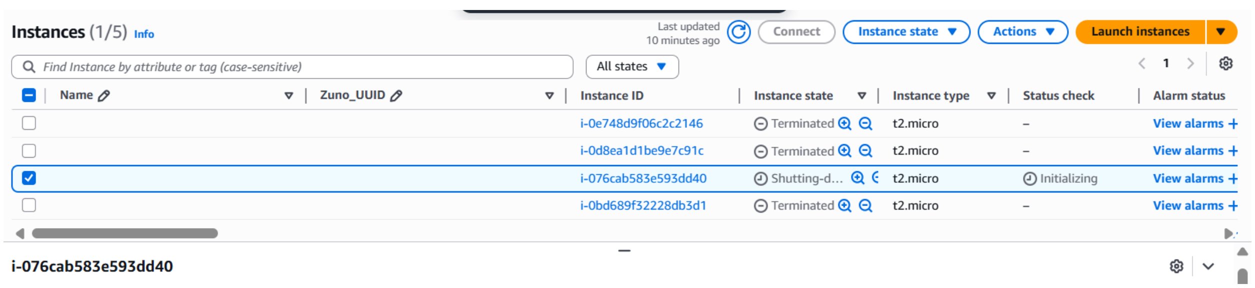
Create EventBridge Rule



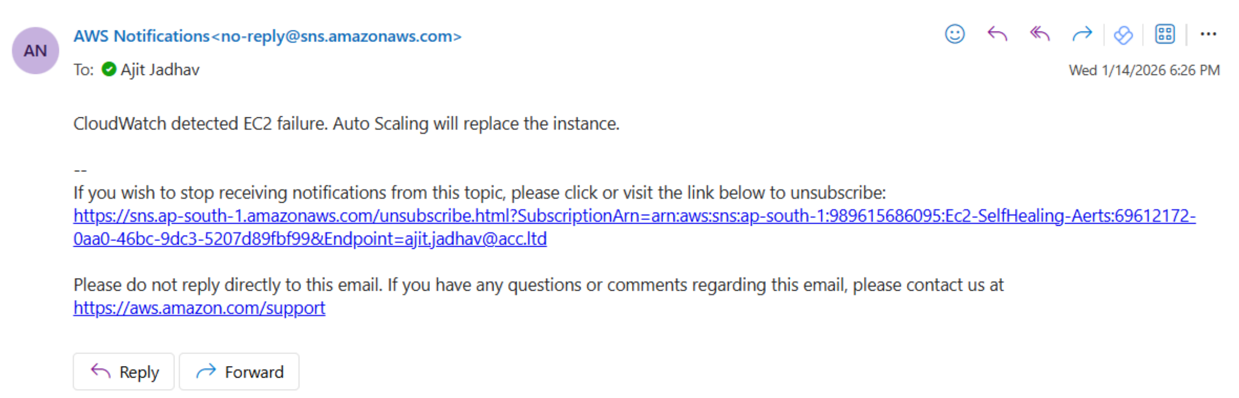




Now we stop our running instance



Notification Via SNS



New instance Launched By ASG